

Learn HTML

Lesson 8: Project: Build a Simple Webpage

Project: Build a Simple Webpage – Study Notes

Key Concepts

1. **HTML document structure** – `<!DOCTYPE>`, `<html>`, `<head>`, `<body>` tags.
2. **Semantic elements** – `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>` for logical page layout.
3. **Embedding media** – `<img>` for pictures, `<a>` for hyperlinks, and `<form>` elements for user input.
4. **Attributes & accessibility** – `src`, `alt`, `href`, `type`, `name`, `placeholder`, `required`, and ARIA basics.
5. **Basic CSS styling (optional but recommended)** – linking an external stylesheet with `<link>` and using simple selectors to improve readability.

Summary

In this project you will bring together the HTML tags you've learned so far to construct a personal profile webpage. The page will contain a heading with your name, a short bio, a portrait image, links to your social media profiles, and a contact form that visitors can use to send you a message. By following the step by step markup, you'll see how each element fits into the overall document hierarchy and how attributes give extra information to browsers and assistive technologies.

The exercise reinforces the idea that a web page is built from **small, reusable pieces**: headings (`<h1>` `<h6>`), paragraphs (`<p>`), lists (`<ul>`, `<li>`), images (`<img>`), links (`<a>`), and form controls (`<input>`, `<textarea>`, `<button>`). When these pieces are organized inside semantic containers (`<header>`, `<section>`, `<footer>`), the markup becomes easier to read, maintain, and style with CSS.

After completing the project you'll have a functional, accessible HTML file that can be opened in any browser and later expanded with CSS or JavaScript.

Important Points

- **Doctype & language** – Start every page with `<!DOCTYPE html>` and add `lang="en"` (or your language) to `<html>` for accessibility and SEO.
- **Head elements** – Use `<meta charset="UTF-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1.0">`, and a descriptive `<title>`.
- **Image accessibility** – Always provide an `alt` attribute; it describes the image for screen readers and when the image fails to load.
- **Link security** – When linking to external sites, add `target="_blank"` **and** `rel="noopener noreferrer"` to prevent tab hijacking.
- **Form basics** –

- Wrap inputs in a `<form>` with an appropriate `action` (or `#` for a demo).
- Use `method="post"` for real submissions.
- Include `name` attributes so server side scripts can identify each field.
- Mark required fields with `required`.
 - **Semantic markup** – Prefer `<section>` for distinct page parts and `<article>` for self contained content (e.g., a bio).
 - **Validation** – Run your HTML through the W3C validator to catch missing tags or attribute errors.

Examples

Example 1 – Minimal Profile Page

```

<<html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jane Doe – Portfolio</title>
</head>
<body>
<header>
  <h1>Jane Doe</h1>
  <p>Front end developer & UI enthusiast</p>
</header>
<section id="about">
  <h2>About Me</h2>
  
  <p>Hello! I love turning ideas into beautiful, responsive websites.</p>
</section>
<section id="links">
  <h2>Find Me Online</h2>
  <ul>
    <li><a href="https://github.com/janedoe" target="_blank"
      rel="noopener noreferrer">GitHub</a></li>
    <li><a href="https://linkedin.com/in/janedoe" target="_blank"
      rel="noopener noreferrer">LinkedIn</a></li>
  </ul>
</section>
<section id="contact">
  <h2>Contact Me</h2>
  <form action="#" method="post">

```

```

    <label for="name">Name:</label><br>
    <input type="text" id="name" name="name" placeholder="Your name" required><br><br>
    <label for="email">Email:</label><br>
    <input type="email" id="email" name="email" placeholder="you@example.com" required><br><br>
    <label for="msg">Message:</label><br>
    <textarea id="msg" name="msg" rows="4" placeholder="Your message"></textarea><br><br>
    <button type="submit">Send</button>
  </form>
</section>
<footer>
  <p>&copy; 2026 Jane Doe</p>
</footer>
</body>
</html>

```

***Explanation*:** This file shows the full page skeleton. The `<header>` introduces the person, `<section>` elements separate content, and the `<form>` collects visitor data. All links open in a new tab safely, and the image includes an `alt` description.

Example 2 – Adding a Simple External CSS File

****style.css****

````css`

```

body {font-family: Arial, sans-serif; line-height: 1.6; margin: 0; padding: 0 1rem;}
header, footer {background:#f4f4f4; text-align:center; padding:1rem;}
section {margin:2rem 0;}
img {border-radius:50%; display:block; margin:auto;}
ul {list-style:none; padding:0;}
li {display:inline; margin-right:1rem;}
button {background:#0066cc; color:#fff; border:none; padding:.5rem 1rem; cursor:pointer;}
button:hover {background:#004999;}

```

`````

****HTML (add inside `<head>`):****

````html`

```

<link rel="stylesheet" href="style.css">

```

`````

***Explanation*:** Linking a stylesheet keeps HTML clean and lets you style multiple elements at once. The CSS above gives the page a modern, centered look with minimal code—perfect for a beginner project.

Quick Review

1. ****What tag defines the language of the document and why is it important?****
2. ****Name two semantic elements you would use to separate a personal bio from a contact form.****

3. **Why must every `<img>` tag have an `alt` attribute?**
4. **What attributes should you add to an external link to open it safely in a new tab?**
5. **In a form, why is the `name` attribute essential for each input element?**

Further Reading

- **HTML Semantics** – deeper dive into `<main>`, `<aside>`, `<figure>`, and `<figcaption>`.
- **Responsive Design Basics** – using the viewport meta tag and simple media queries.
- **Accessibility (a11y) Fundamentals** – ARIA roles, keyboard navigation, and color contrast.
- **CSS Layout** – Flexbox and Grid for more complex page structures.
- **Form Handling** – connecting a form to a back end service (e.g., Formspree, Netlify Forms) or using JavaScript for client side validation.

